



UNIVERSITY OF CHITTAGONG

Department of Computer Science and Engineering

Project Report : Smart Voice Assistant

Course title: Microprocessor and Embedded Systems Lab

Course Code: CSE 416

Submitted to:

Dr. M. Jamshed Alam Patwary

Department of Computer Science and Engineering

University of Chittagong

Submitted by:

Name: Sheikh Tawsif Bin Aftab

ID: 24701026

Name: Safwan Ahmed Rayed

ID: 24701066

Name: Syed Mehraz Monwar

ID: 24701042

Name: Sanjida Mahi

ID: 24701009

Session: 2023-2024 B.Sc. Engineering (4th Semester)

Department of Computer Science and Engineering

University of Chittagong

Contents

Abstract.....	i
1. Introduction.....	1
2. Components Used.....	2
2.1 ESP32-S3 DevKitC-1 N16R8.....	2
2.2 ST7789 TFT Display	3
2.3 MAX4466 Microphone.....	3
2.4 TTP223B Capacitive Touch Sensor	4
2.5 Tactile Button.....	4
2.6 HW-104 / PAM8403 Audio Amplifier	5
2.7 Speaker.....	5
2.8 XL6009 Boost Converter	5
2.9 Windows PC/Laptop	6
2.10 Android Phone with Tool Neuron	6
2.11 Additional Components.....	6
2.12 Software Used.....	6
3. Circuit Assembly and Design.....	7
3.1 System Architecture.....	7
3.2 Pin Connections	8
3.3 Display Connections	8
3.4 Microphone Connection.....	8
3.5 Audio Amplifier Connection.....	9
3.6 Power Supply	9
4. Working Principle	10
4.1 Device Initialization	10
4.2 Display Modes	11
4.3 Voice Interaction	11

4.4 AI Server Processing.....	12
4.5 Audio Playback	12
4.6 Configuration Portal.....	13
5. Program Implementation	14
5.1 ESP32 Firmware	14
5.2 Windows AI Server	15
6. Output and Testing	18
6.1 Experimental Setup	18
6.2 Testing Procedure.....	19
6.3 Test Results	20
6.4 Known Limitations	21
6.5 Server Output	22
7. Advantages.....	24
8. Conclusion	25
Future Enhancements.....	26
Acknowledgment.....	27
References	28

Abstract

This project presents the design and implementation of an ESP32-based AI Voice Assistant, a compact desk companion device capable of voice interaction, real-time clock display, weather information, and local AI-powered conversations. The system utilizes an ESP32-S3 DevKitC-1 microcontroller as the central processing unit, integrated with a 1.3-inch ST7789 TFT display for visual output, a MAX4466 microphone for audio input, and a PAM8403 audio amplifier for speech output.

The device captures user voice commands through the microphone, transmits the audio to a local Windows-based AI server via Wi-Fi, and receives transcribed text responses along with synthesized speech audio. The server pipeline includes speech-to-text conversion, text generation, and text-to-speech synthesis through Tool Neuron running on an Android phone. The ESP32 firmware handles audio playback, display rendering with multiple screen modes (digital clock, analog clock, weather, system status, and AI visualizer), and user input through a capacitive touch sensor and tactile button.

The project demonstrates a practical embedded systems application combining IoT connectivity, local AI inference, audio processing, and graphical user interface design. Testing results confirm successful voice capture, AI response generation, and audio playback functionality. The system offers a low-cost, privacy-focused alternative to cloud-dependent voice assistants, with the AI workflow running through the Windows PC server and Tool Neuron phone on the local network.

1. Introduction

Voice assistants have become an integral part of modern human-computer interaction, enabling users to control devices, access information, and perform tasks through natural speech. Commercial voice assistants such as Amazon Alexa, Google Assistant, and Apple Siri rely heavily on cloud connectivity, raising concerns about privacy, latency, and dependency on internet services. This project addresses these limitations by developing a local-first AI voice assistant using embedded systems technology.

The Smart Voice Assistant (codenamed "Smart Voice Assistant") is a compact, standalone device built around the ESP32-S3 microcontroller. The device provides the following key features:

- Real-time clock and weather display on a 240x240 TFT color screen
- Voice-activated AI conversation through local server processing
- Multiple display modes: digital clock, analog clock, weather, system status, and AI visualizer
- Touch and button-based user interaction

- Prototype operation using USB power for the ESP32 and an XL6009-powered amplifier

The project architecture follows a client-server model where the ESP32 acts as the client device handling audio capture, playback, and display rendering, while a local Windows PC runs the AI server providing speech recognition, natural language processing, and speech synthesis services. This approach avoids paid cloud API dependency during normal operation and keeps the voice assistant workflow inside the local network used by the ESP32, PC, and phone.

2. Components Used

The following hardware and software components were used to develop the Smart Voice Assistant.

No.	Component	Quantity	Purpose
1	ESP32-S3 DevKitC-1 N16R8	1	Main microcontroller with Wi-Fi
2	ST7789 TFT Display 240x240	1	Main display screen
3	MAX4466 Microphone Module	1	Audio input capture
4	TTP223B Touch Sensor	1	Touch/page input
5	4-Pin Tactile Button	1	AI mode and recording control
6	HW-104 / PAM8403 Amplifier	1	Speaker driver using right input R
7	XL6009 Boost Converter	1	5V amplifier power
8	Speaker	1	Audio output
9	Red LED + Resistor	1	Status indicator
10	Breadboard + Jumper Wires	1 set	Circuit prototyping
11	Windows PC/Laptop	1	Runs FastAPI bridge server
12	Android Phone	1	Runs Tool Neuron STT/LLM/TTS

2.1 ESP32-S3 DevKitC-1 N16R8

The ESP32-S3 DevKitC-1 is the main microcontroller of the AI Voice Assistant. It features a dual-core Xtensa LX7 processor running at 240 MHz, 2.4 GHz Wi-Fi, Bluetooth 5 (LE), 16 MB of flash memory, and 8 MB of PSRAM. The board provides multiple GPIO pins for connecting peripherals and supports USB-C for programming and power. The large memory capacity makes it suitable for handling audio buffers, graphics rendering, and Wi-Fi communication simultaneously.

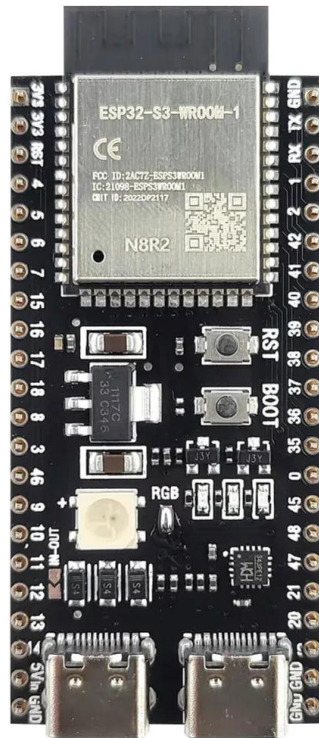


Figure 1: ESP32-S3 DevKitC-1 N16R8 Development Board

2.2 ST7789 TFT Display

The 1.3-inch ST7789 TFT display provides a 240x240 pixel full-color screen for the user interface. It communicates with the ESP32 via SPI interface and features a 7-pin configuration without a chip select (CS) line. The display supports hardware reset tied to the ESP32 EN pin and uses Arduino_GFX library for rendering. The screen is mounted upside down in the enclosure, requiring a 180-degree rotation in firmware.



Figure 2: ST7789 1.3-inch 240x240 TFT Display Module

2.3 MAX4466 Microphone

The MAX4466 is an electret microphone amplifier module with adjustable gain. It provides analog audio output to the ESP32's ADC pin for voice capture. The module operates at 3.3V and includes a built-in preamplifier with a trimmer potentiometer for sensitivity adjustment. While the analog microphone introduces some noise, it is sufficient for prototype development. Future iterations may upgrade to an I2S digital microphone such as the INMP441 for improved audio quality.



Photo by ElectroPeak

Figure 3: MAX4466 Electret Microphone Amplifier Module

2.4 TTP223B Capacitive Touch Sensor

The TTP223B capacitive touch sensor provides touch input for cycling through display pages. When touched, the sensor outputs a HIGH signal to the ESP32 GPIO pin. The sensor operates at 3.3V and features built-in debouncing and auto-calibration for reliable touch detection.



Figure 4: TTP223B Capacitive Touch Sensor Module

2.5 Tactile Button

A 4-pin tactile push button serves as the primary user input for activating AI mode and controlling recording. Short presses trigger mode transitions, while long presses (900ms) enter the Wi-Fi setup portal. The button connects between GPIO6 and GND with internal pullup enabled.



Figure 5: 4-Pin Tactile Push Button

2.6 HW-104 / PAM8403 Audio Amplifier

The HW-104 (PAM8403) is a 3W class-D audio amplifier module that drives the speaker. In the current prototype, ESP32 GPIO4 is connected to the right channel input R of the amplifier, while the left input L is disconnected. The PAM8403 is powered from a 5V XL6009 boost converter and provides bridge-tied speaker outputs that must not be connected to ground.

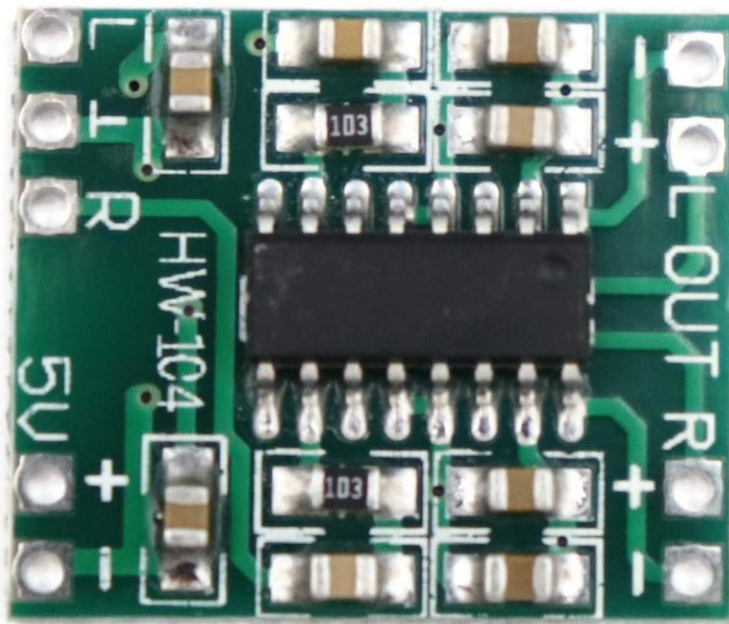


Figure 6: HW-104 / PAM8403 Audio Amplifier Module

2.7 Speaker

A small speaker provides audio output for AI voice responses and notification beeps. It connects to the right output pair of the HW-104/PAM8403 amplifier: R+ and R-. Neither speaker output should be connected to ground.



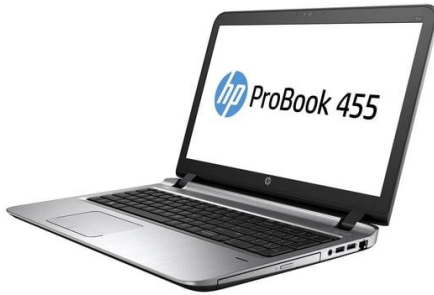
Figure 7: Speaker used for voice output



2.8 XL6009 Boost Converter

The XL6009 boost converter supplies regulated 5V power to the HW-104/PAM8403 amplifier. Its output must be checked with a multimeter before connecting the amplifier, because excessive voltage can damage the module or destabilize the ESP32 through shared ground noise.

Figure 8: XL6009 Boost Converter Module



2.9 Windows PC/Laptop

The Windows PC runs the FastAPI server that receives audio from the ESP32, forwards AI requests to Tool Neuron on the phone, saves uploaded recordings, and serves generated speech audio back to the ESP32.

Figure 9: Windows PC/Laptop used for local server operation

2.10 Android Phone with Tool Neuron

An Android phone runs Tool Neuron and provides the OpenAI-compatible endpoints used for speech-to-text, AI text response generation, and text-to-speech. In testing, the phone server was reached over the local network from the Windows PC server.



Figure 10: Android phone used to run Tool Neuron

2.12 Additional Components

Additional components include a red rectangular LED with current-limiting resistor (220 ohm to 1k ohm) for status indication, a solderless breadboard for prototyping, male-to-male and male-to-female jumper wires for connections, and a USB-C cable for programming and debugging.

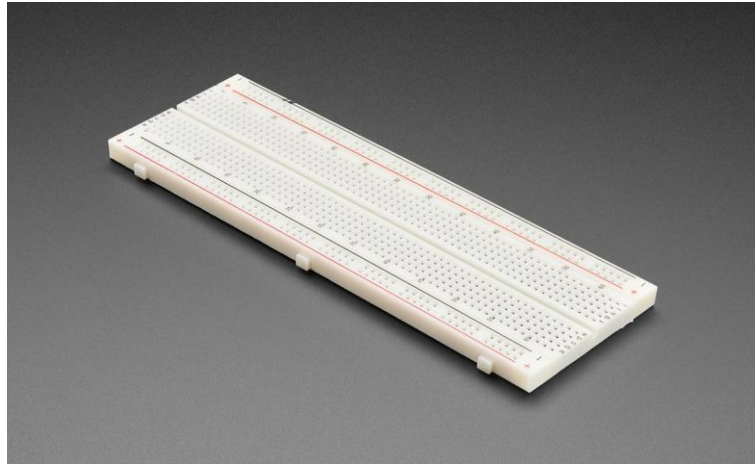


Figure 12: Solderless Breadboard for Prototyping

2.13 Software Used

The following software tools and frameworks were used during development:

- Arduino IDE / Arduino CLI: Used to write, compile, and upload the ESP32 firmware
- Python 3.11+: Used for the AI server backend
- FastAPI + Uvicorn: Web framework for the local AI server
- Tool Neuron: Android-based AI service for speech-to-text, text generation, and text-to-speech
- FastAPI/Uvicorn: Windows PC server for ESP32 audio upload and response delivery
- Tool Neuron: Android-based AI service for STT/LLM/TTS
- Arduino_GFX Library: Display driver for the ST7789 screen

3. Circuit Assembly and Design

The Smart Voice Assistant is designed around the ESP32-S3 DevKitC-1, which serves as the central controller. The ESP32 receives input from the microphone, touch sensor, and button; processes the collected data; and controls the display, audio amplifier, and status LED accordingly.

3.1 System Architecture

The system operates according to the following architecture:

User speaks / presses button --> ESP32 firmware --> Raw audio POST to Windows Server --> Speech-to-Text --> LLM Text Generation --> Text-to-Speech --> WAV audio --> ESP32 playback --> Speaker output

In parallel, the ESP32 maintains Wi-Fi connection, updates the clock via NTP, fetches weather data from wttr.in, and renders the appropriate display screen.

3.2 Pin Connections

The following table summarizes the hardware connections used in the project:

Component	ESP32-S3 Pin
Display SCK/SCL	GPIO12
Display MOSI/SDA	GPIO11
Display DC	GPIO9
Display CS	None (-1 in firmware)
Display RES	ESP32 EN/RST (physical)
Microphone OUT	GPIO1
HW-104 R input	GPIO4
Touch Sensor SIG	GPIO5

Component	ESP32-S3 Pin
Button	GPIO6
Test LED	GPIO2

3.3 Display Connections

The ST7789 display connects to the ESP32 via SPI:

- VCC --> 3V3
- GND --> GND
- SCL --> GPIO12
- SDA --> GPIO11
- RES --> ESP32 EN/RST (physically tied)
- DC --> GPIO9
- BLK --> disconnected (backlight always on)

Important: This display module has no CS pin. The firmware uses CS=-1 configuration. The display only works reliably when RES is physically tied to the ESP32 reset pin.

3.4 Microphone Connection

The MAX4466 microphone module connects as follows:

- VCC --> 3V3
- GND --> GND
- OUT --> GPIO1 (ADC)

The microphone wires should be kept short and powered from the 3.3V rail to minimize noise.

3.5 Audio Amplifier Connection

The HW-104/PAM8403 amplifier uses the right channel input:

- GPIO4 --> R input

-
- XL6009 OUT+ --> 5V+
- XL6009 OUT- --> 5V-
- G --> ESP32 GND
- L --> disconnected
- R+ and R- --> Speaker

Note: Do not connect either speaker output wire to ground, as the PAM8403 uses bridge-tied outputs.

3.6 Power Supply

The ESP32 is powered through USB during programming and testing. The HW-104/PAM8403 amplifier is powered from the XL6009 boost converter adjusted to 5V. ESP32 ground, XL6009 OUT-, and HW-104 ground must be connected together as common ground.

4. Working Principle

The Smart Voice Assistant operates through a series of interconnected stages that handle user input, audio processing, AI communication, and output rendering. The overall operation can be divided into the following stages:

4.1 Device Initialization

Upon power-up, the ESP32 performs the following initialization sequence:

- Starts serial communication at 115200 baud
- Generates a unique device ID from the efuse MAC address
- Configures GPIO pins: touch input, button with pullup, LED output
- Sets ADC resolution to 12 bits with 11dB attenuation
- Configures PWM output on GPIO4 for audio at 40kHz carrier frequency
- Initializes the ST7789 display via Arduino_GFX library
- Loads saved configuration from NVS (Wi-Fi credentials, server URL, weather city)

- Attempts Wi-Fi connection and syncs time via NTP
- Fetches initial weather data from wttr.in

4.2 Display Modes

The device supports multiple display modes that the user can cycle through:

- Digital Clock: Dot-matrix style clock with weather summary
- Analog Clock: Traditional analog clock face with hour and minute hands
- Weather Screen: Animated weather icon with temperature and conditions
- System Screen: Wi-Fi status, IP address, server URL, mic level, free RAM/flash
- AI Visualizer: Animated bouncing dots that respond to microphone input

4.3 Voice Interaction

The voice interaction follows a three-press button sequence:

- First press: Enters AI mode and displays the AI visualizer
- Second press: Starts recording audio from the microphone (up to 5 seconds)
- Third press: Stops recording and sends the audio to the server

Recording automatically stops after 1.2 seconds of silence. The ESP32 captures 8kHz 8-bit unsigned PCM audio and sends it as a raw binary POST to the server's /api/voice-message endpoint.

4.4 AI Server Processing

The local AI server (running on Windows) processes the voice data through the following pipeline:

- Receives raw audio upload from ESP32
- Wraps the raw PCM data as a WAV file
- Sends audio to Tool Neuron STT on the Android phone

- Sends transcribed text to the Tool Neuron chat model
- Sends response text to Tool Neuron TTS
- Converts generated audio to ESP32-friendly 8kHz mono 8-bit WAV
- Returns text response and audio URL to ESP32

4.5 Audio Playback

The ESP32 receives the audio URL, downloads the WAV file via HTTP streaming, and plays it back using PWM on GPIO4 into the PAM8403 amplifier right input. The amplifier drives the speaker through the R+ and R- output pair. The streaming playback reads WAV data directly from the WiFiClient stream to minimize memory usage.

4.6 Configuration Portal

A long button press (900ms) activates the Wi-Fi configuration portal. The ESP32 creates an access point named "AI_Buddy_Setup" with password "12345678". Users connect to this AP and navigate to 192.168.4.1 to configure Wi-Fi credentials, local server URL, weather city, and startup page preference.

5. Program Implementation

The software component consists of two main parts: the ESP32 firmware (Arduino C++) and the Windows AI Server (Python/FastAPI).

5.1 ESP32 Firmware

The Arduino firmware handles all hardware interactions including display rendering, audio capture/playback, button/touch input, and Wi-Fi communication. Key features of the firmware include:

- Display rendering using Arduino_GFX library with ST7789 driver
- Custom 7-segment and dot-matrix font rendering for the clock
- Microphone sampling at 8kHz using timed ADC reads
- PWM-based audio playback with WAV header parsing

- HTTP client for server communication and audio download
- NTP time synchronization and weather API integration
- Web server for the configuration portal
- Partial screen redraw to minimize flicker

The firmware is built using Arduino CLI with the following board configuration:

```
esp32:esp32:esp32s3:CDCCOnBoot=cdc,PartitionScheme=huge_app,PSRAM=opi
```

Build statistics from the latest firmware compilation:

- Program storage: approximately 1,186,094 bytes (37%)
- Dynamic memory: approximately 49,576 bytes (15%)
- Free RAM: approximately 278,104 bytes

5.2 Windows AI Server

The AI server is a FastAPI application that provides the following endpoints:

- GET /api/health: Returns server status and configuration
- POST /api/voice-message: Accepts audio upload, processes through STT/LLM/TTS pipeline
- POST /api/default-message: Generates a default greeting message
- GET /api/tool-neuron-models: Lists available AI models

The server primarily forwards AI work to Tool Neuron and keeps fallbacks only for debugging:

- Text Generation: Tool Neuron chat model, with local fallback only for debugging
- Speech-to-Text: Tool Neuron STT model
- Text-to-Speech: Tool Neuron TTS model, with Windows SAPI fallback only for debugging

Key server components include:

```
# Server configuration from .env file
TOOL_NEURON_BASE_URL=http://192.168.207.57:11434/v1
PUBLIC_BASE_URL=http://192.168.207.192:3000
```

The server converts all generated audio to ESP32-compatible format: 8kHz, mono, unsigned 8-bit WAV.

6. Output and Testing

After completing the hardware assembly and software implementation, the Smart Voice Assistant was tested under different operating conditions to evaluate its performance, accuracy, and reliability.

6.1 Experimental Setup

The complete prototype was assembled on a solderless breadboard with the ESP32-S3 DevKitC-1, ST7789 display, MAX4466 microphone, PAM8403 amplifier, speaker, and supporting components. The ESP32 was connected to a laptop via USB-C for programming, and the AI server ran on the same laptop connected to the local Wi-Fi network.

6.2 Testing Procedure

The following tests were performed:

- Display Test: Verified all display modes render correctly without flicker
- Button Input Test: Confirmed short press, long press, and mode cycling
- Touch Input Test: Verified page cycling via touch sensor
- Microphone Test: Recorded audio and verified WAV file creation on server
- Wi-Fi Connection Test: Verified stable connection and reconnection
- Server Communication Test: Confirmed health endpoint and voice message pipeline
- Audio Playback Test: Verified startup beep and server-generated speech playback
- Configuration Portal Test: Verified AP mode and settings persistence

6.3 Test Results

Test Scenario	Result	Status
Display initialization	All modes render correctly	Pass
Button short press	Mode transitions work	Pass

Test Scenario	Result	Status
Button long press	Setup portal opens	Pass
Touch sensor	Page cycling works	Pass
Microphone recording	Audio captured and saved	Pass
Wi-Fi connection	Stable connection	Pass
Server health check	All services configured	Pass
Voice message pipeline	Transcript + response + audio	Pass
Audio playback	Startup beep path and generated audio file path tested; final speaker clarity depends on amp voltage and wiring	Hardware-dependent
Configuration portal	Settings saved to NVS	Pass

6.4 Known Limitations

During testing, the following limitations were identified:

- The MAX4466 microphone produces audible noise in recordings
- Whisper tiny model occasionally hallucinates repeated phrases on noisy input
- PWM audio output through the analog amplifier can still be noisy and depends strongly on amplifier voltage, grounding, and speaker wiring
- Touch sensor can be unreliable during AI mode debugging

6.5 Server Output

The AI server successfully generates speech responses that play through the device speaker. The health endpoint reports all services configured correctly, and the voice message pipeline completes with transcript, response text, and audio URL in the response.

7. Advantages

The Smart Voice Assistant offers several advantages over commercial cloud-based voice assistants:

- Local-network operation: voice data is handled by the ESP32, Windows PC server, and Android phone running Tool Neuron during normal operation
- Low Cost: Uses affordable, readily available components (total BOM under \$30)
- No paid API requirement for normal testing: uses Tool Neuron on the Android phone
- Customizable: Fully open-source firmware and server software
- Modular prototype: ESP32, display, phone server, PC server, and amplifier can be tested independently
- Expandable: Modular design allows easy hardware upgrades
- Educational: Demonstrates embedded systems, IoT, and AI integration concepts

8. Conclusion

The Smart Voice Assistant project successfully demonstrates a practical, privacy-focused voice assistant built using embedded systems technology. The device combines an ESP32-S3 microcontroller with a color display, microphone, and audio amplifier to create a compact desk companion capable of voice-based AI interaction.

The project achieves its primary objectives: local voice capture, server-based AI processing, audio playback, and graphical user interface rendering. The modular client-server architecture allows the ESP32 to handle real-time tasks (audio, display, input) while delegating AI processing to a local PC server running open-source models.

Testing confirms that all major subsystems function correctly: the display renders multiple screen modes, the microphone captures voice input, the server generates appropriate AI responses, and the speaker plays back synthesized speech. The configuration portal provides a user-friendly method for device setup.

This project serves as a foundation for further development in embedded AI applications, demonstrating that capable voice assistants can be built without cloud dependencies using affordable, open-source components.

Future Enhancements

The current prototype successfully achieves its core objectives. Several improvements can be incorporated in future versions:

- Upgrade to INMP441 I2S digital microphone for improved audio quality
- Use a digital I2S audio amplifier in a future version for cleaner speaker output
- Design a custom PCB to replace the breadboard prototype
- Add a 3D-printed enclosure for a finished product appearance
- Implement wake-word detection for hands-free activation
- Add support for multiple languages in STT and TTS
- Integrate with smart home devices for voice-controlled automation
- Implement continuous conversation mode without repeated button presses
- Add small offline preset commands on the ESP32-S3 for simple non-AI responses
- Develop a mobile app for remote configuration and monitoring

Acknowledgment

The authors would like to express their sincere gratitude to the Department of Computer Science and Engineering for providing the opportunity to undertake this project. Special thanks are extended to the course instructor for their valuable guidance, encouragement, and technical support throughout the development of this project.

The successful completion of this work would not have been possible without the support of our teachers, classmates, and the facilities provided by the university. Their assistance and constructive feedback contributed significantly to the successful implementation of the Smart Voice Assistant.

References

1. Espressif Systems. ESP32-S3 Datasheet. Available at: <https://www.espressif.com/>

2. Arduino. Arduino_GFX Library Documentation. Available at:
https://github.com/moononournation/Arduino_GFX
3. FastAPI Documentation. Available at: <https://fastapi.tiangolo.com/>
4. Tool Neuron Android application and OpenAI-compatible local API interface.
5. FastAPI documentation. Available at: <https://fastapi.tiangolo.com/>
6. ST7789 Display Controller Datasheet.
7. MAX4466 Microphone Amplifier Datasheet. Maxim Integrated.
8. PAM8403 Audio Amplifier Datasheet. Diodes Incorporated.
10. Python Software Foundation. Python 3 Documentation. Available at:
<https://docs.python.org/3/>